```
# Import required library
import pandas as pd

# Upload dataset in Google Colab
from google.colab import files

uploaded = files.upload()

# Get uploaded file name
filename = list(uploaded.keys())[0]

# Read the dataset
df = pd.read_csv(filename)

# Display first 5 records
print("Dataset loaded successfully!")
print(df.head())

# Display dataset dimensions
print("\nDataset Shape:")
print(df.shape)

# Display column names
print("\nDataset Columns:")
print(df.columns.tolist())
```

```
Choose Files Churn_Modelling.csv
Churn_Modelling.csv(text/csv) - 684858 bytes, last modified: 10/6/2026 - 100% done
Saving Churn_Modelling.csv to Churn_Modelling.csv
Dataset loaded successfully!
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited
0        101348.88       1
1        112542.58       0
2        113931.57       1
3         93826.63       0
4         79084.10       0

Dataset Shape:
(10000, 14)

Dataset Columns:
['RowNumber', 'CustomerId', 'Surname', 'CreditScore', 'Geography', 'Gender', 'Age', 'Tenure', 'Balance
```

```
# Check dataset information
print("Dataset Information:")
df.info()

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Handle missing values
numeric_columns = df.select_dtypes(include='number').columns
categorical_columns = df.select_dtypes(include='object').columns
```

```
for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

# Check duplicate records
print("\nDuplicates before removal:")
print(df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()

print("\nDuplicates after removal:")
print(df.duplicated().sum())

# Handle outliers using IQR method
for col in numeric_columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    df = df[
        (df[col] >= lower_limit) &
        (df[col] <= upper_limit)
    ]

# Encode categorical variables
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

for col in categorical_columns:
    df[col] = encoder.fit_transform(df[col].astype(str))

print("\nPreprocessed Dataset:")
print(df.head())

print("\nFinal Shape:")
print(df.shape)
```

```
 3   CreditScore      10000 non-null  int64
 4   Geography        10000 non-null  object
 5   Gender           10000 non-null  object
 6   Age              10000 non-null  int64
 7   Tenure           10000 non-null  int64
 8   Balance          10000 non-null  float64
 9   NumOfProducts    10000 non-null  int64
 10  HasCrCard        10000 non-null  int64
 11  IsActiveMember   10000 non-null  int64
 12  EstimatedSalary  10000 non-null  float64
 13  Exited           10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB

Missing Values:
RowNumber          0
CustomerId         0
Surname            0
CreditScore        0
```

```
Exited             0
dtype: int64

Duplicates before removal:
0

Duplicates after removal:
0

Preprocessed Dataset:
   RowNumber  CustomerId  Surname  CreditScore  Geography  Gender  Age  \
1          2    15647311     1035          608          2       0   41
3          4    15701354      259          699          0       0   39
4          5    15737888     1599          850          2       0   43
6          7    15592531      161          822          0       1   50
8          9    15792365     1009          501          0       1   44

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
1       1   83807.86              1          0               1
3       1       0.00              2          0               0
4       2  125510.82              1          1               1
6       7       0.00              2          1               1
8       4  142051.07              2          0               1

   EstimatedSalary  Exited
1        112542.58       0
3         93826.63       0
4         79084.10       0
6         10062.80       0
8         74940.50       0
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Display first records
print("First 5 Records:")
print(df.head())

# Descriptive statistics
print("\nDescriptive Statistics:")
print(df.describe())

# Dataset information
print("\nDataset Information:")
df.info()

# Histograms
df.hist(figsize=(12, 8))
plt.tight_layout()
plt.show()

# Boxplots
plt.figure(figsize=(12, 6))
sns.boxplot(data=df)
plt.xticks(rotation=45)
plt.title("Boxplot of Dataset Variables")
plt.show()
```

```
First 5 Records:
   RowNumber  CustomerId  Surname  CreditScore  Geography  Gender  Age  \
1          2    15647311     1035          608          2       0   41
3          4    15701354      259          699          0       0   39
4          5    15737888     1599          850          2       0   43
6          7    15592531      161          822          0       1   50
8          9    15792365     1009          501          0       1   44

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
1       1   83807.86              1          0               1
3       1       0.00              2          0               0
4       2  125510.82              1          1               1
6       7       0.00              2          1               1
8       4  142051.07              2          0               1

   EstimatedSalary  Exited
1        112542.58       0
3         93826.63       0
4         79084.10       0
6         10062.80       0
8         74940.50       0

Descriptive Statistics:
          RowNumber    CustomerId      Surname  CreditScore    Geography  \
count   7677.000000  7.677000e+03  7677.000000  7677.000000  7677.000000
mean    5022.567149  1.569113e+07  1327.858017   651.532500     0.731536
std     2893.347848  7.164727e+04   740.697722    95.681256     0.845219
min        2.000000  1.556570e+07     0.000000   405.000000     0.000000
25%     2514.000000  1.562870e+07   689.000000   584.000000     0.000000
50%     5046.000000  1.569186e+07  1361.000000   653.000000     0.000000
75%     7521.000000  1.575296e+07  1978.000000   717.000000     2.000000
max    10000.000000  1.581569e+07  2566.000000   850.000000     2.000000

            Gender          Age       Tenure        Balance  NumOfProducts  \
count  7677.000000  7677.000000  7677.000000    7677.000000    7677.000000
mean      0.573010    36.193826     5.035561   72870.208446       1.545786
std       0.494673     8.013258     2.876186   62838.720574       0.508802
min       0.000000    18.000000     0.000000       0.000000       1.000000
25%       0.000000    31.000000     3.000000       0.000000       1.000000
50%       1.000000    36.000000     5.000000   92224.560000       2.000000
75%       1.000000    41.000000     7.000000  126473.990000       2.000000
max       1.000000    62.000000    10.000000  221532.800000       3.000000

         HasCrCard  IsActiveMember  EstimatedSalary  Exited
count  7677.000000     7677.000000      7677.000000  7677.0
mean      0.706917        0.539664     99848.972807     0.0
std       0.455206        0.498457     57393.876561     0.0
min       0.000000        0.000000        90.070000     0.0
25%       0.000000        0.000000     50906.650000     0.0
50%       1.000000        1.000000     99800.870000     0.0
75%       1.000000        1.000000    148727.420000     0.0
max       1.000000        1.000000    199992.480000     0.0

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 7677 entries, 1 to 9999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RowNumber        7677 non-null   int64
 1   CustomerId       7677 non-null   int64
 2   Surname          7677 non-null   int64
 3   CreditScore      7677 non-null   int64
 4   Geography        7677 non-null   int64
 5   Gender           7677 non-null   int64
 6   Age              7677 non-null   int64
 7   Tenure           7677 non-null   int64
 8   Balance          7677 non-null   float64
 9   NumOfProducts    7677 non-null   int64
 10  HasCrCard        7677 non-null   int64
 11  IsActiveMember   7677 non-null   int64
 12  EstimatedSalary  7677 non-null   float64
 13  Exited           7677 non-null   int64
dtypes: float64(2), int64(12)
memory usage: 899.6 KB
          RowNumber                  CustomerId                     Surname                  CreditScore
```

```
from scipy.stats import pearsonr

# Select numerical columns
numeric_df = df.select_dtypes(include='number')

# Correlation matrix
correlation = numeric_df.corr()

print("Correlation Matrix:")
print(correlation)

# Correlation heatmap
plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()

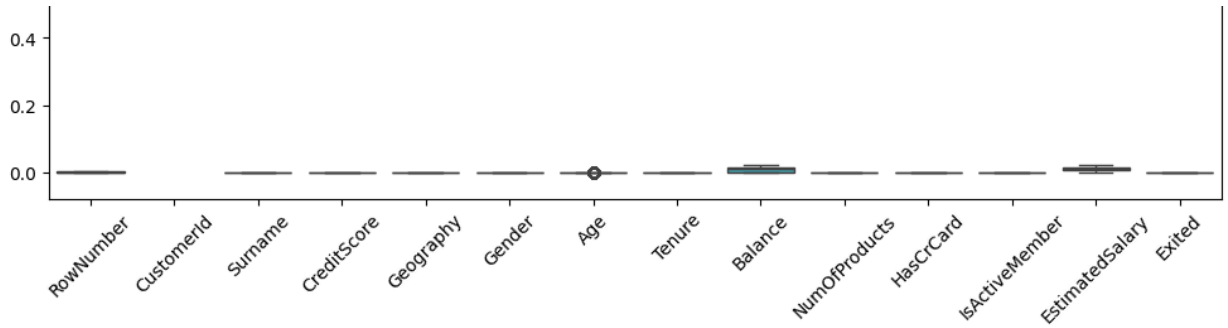
# Pearson correlation between first two numerical variables
columns = numeric_df.columns

if len(columns) >= 2:
    x = numeric_df[columns[0]]
    y = numeric_df[columns[1]]

    correlation_value, p_value = pearsonr(x, y)

    print("\nPearson Correlation:", correlation_value)
    print("P-value:", p_value)

    if p_value < 0.05:
        print("The relationship is statistically significant.")
    else:
        print("The relationship is not statistically significant.")
```



```
Correlation Matrix:
                 RowNumber  CustomerId   Surname  CreditScore  Geography  \
RowNumber         1.000000    0.005613 -0.000873     0.000537  -0.010633
CustomerId        0.005613    1.000000  0.001831     0.004601   0.013955
Surname          -0.000873    0.001831  1.000000     0.001764  -0.020297
CreditScore       0.000537    0.004601  0.001764     1.000000   0.007242
Geography        -0.010633    0.013955 -0.020297     0.007242   1.000000
Gender            0.011351   -0.010180 -0.003412    -0.001064   0.008093
Age               0.006040    0.017139  0.011724    -0.006683   0.033151
Tenure           -0.001523   -0.025680 -0.007453    -0.008403   0.012877
Balance          -0.016605   -0.007739  0.006160     0.008754   0.056642
NumOfProducts     0.001600    0.018324 -0.013565     0.013669   0.006927
HasCrCard         0.003770   -0.011445 -0.006648    -0.000095  -0.007464
IsActiveMember   -0.001006    0.000161 -0.002495     0.024015   0.021413
EstimatedSalary  -0.006744    0.020573  0.007619     0.013878  -0.003033
Exited                 NaN         NaN       NaN          NaN        NaN

                   Gender       Age    Tenure   Balance  NumOfProducts  \
RowNumber        0.011351  0.006040 -0.001523 -0.016605       0.001600
CustomerId      -0.010180  0.017139 -0.025680 -0.007739       0.018324
Surname         -0.003412  0.011724 -0.007453  0.006160      -0.013565
```

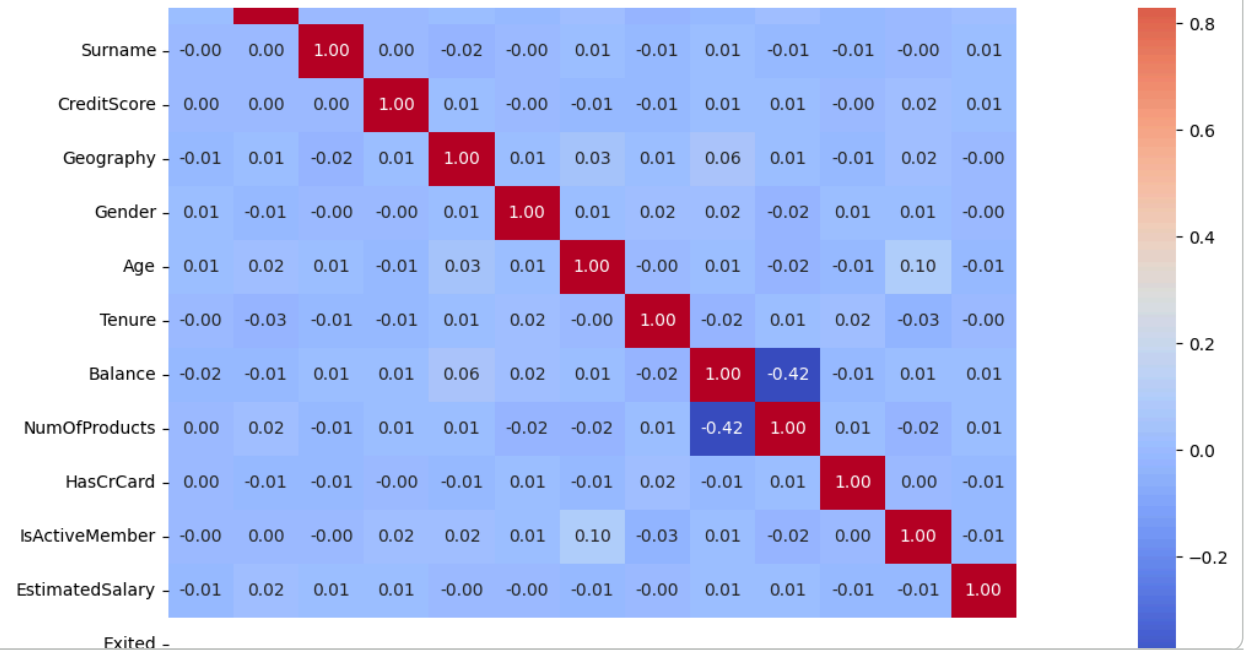
```
import matplotlib.pyplot as plt
import seaborn as sns

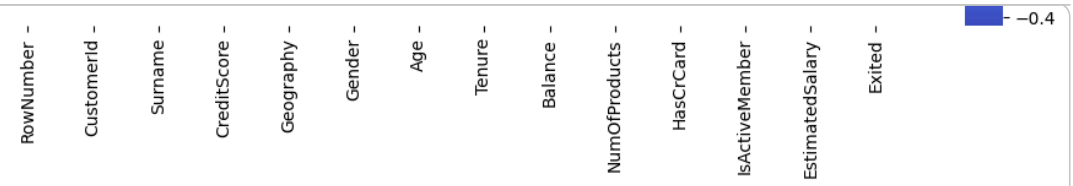
# Correlation heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)
plt.title("Correlation Heatmap")
plt.show()

# Distribution plots
df.hist(figsize=(12, 8))
plt.tight_layout()
plt.show()

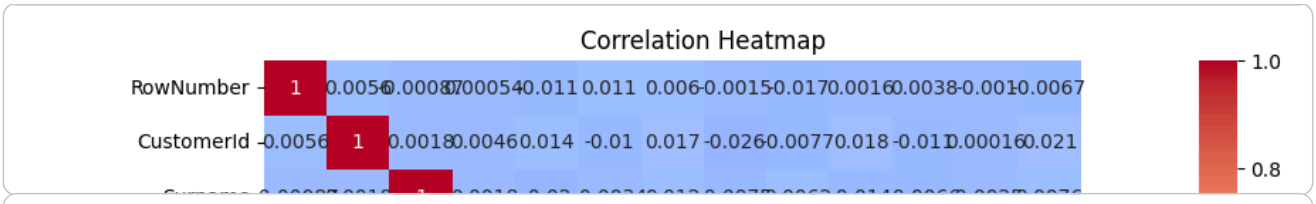
# Count plot for categorical/target column
# Change 'Churn' if your dataset has a different target column

if 'Churn' in df.columns:
    plt.figure(figsize=(6, 4))
    sns.countplot(x=df['Churn'])
    plt.title("Customer Churn Distribution")
    plt.xlabel("Churn")
    plt.ylabel("Number of Customers")
    plt.show()
```






```
Pearson Correlation: 0.005613371863779542
P-value: 0.6228897531115162
The relationship is not statistically significant.
```




```
# Practical Question 6
# Build a Machine Learning Model

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Target column in Churn_Modelling.csv
target_column = "Exited"

# Remove unnecessary columns
df = df.drop(
    columns=["RowNumber", "CustomerId", "Surname"],
    errors="ignore"
)

# Encode categorical columns
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

for col in df.select_dtypes(include="object").columns:
    df[col] = encoder.fit_transform(df[col])

# Separate input and output
X = df.drop(target_column, axis=1)
y = df[target_column]

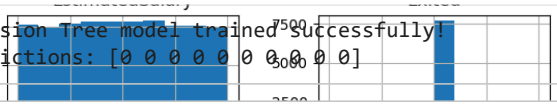
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Create Decision Tree model
model = DecisionTreeClassifier(
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

print("Decision Tree model trained successfully!")
print("Predictions:", y_pred[:10])
```



```
Decision Tree model trained successfully!
Predictions: [0 0 0 0 0 0 0 0 0 0]
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(
    y_test,
    y_pred,
    average='weighted',
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    average='weighted',
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    average='weighted',
    zero_division=0
)

print("===== MODEL EVALUATION =====")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

# Classification report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    zero_division=0
))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

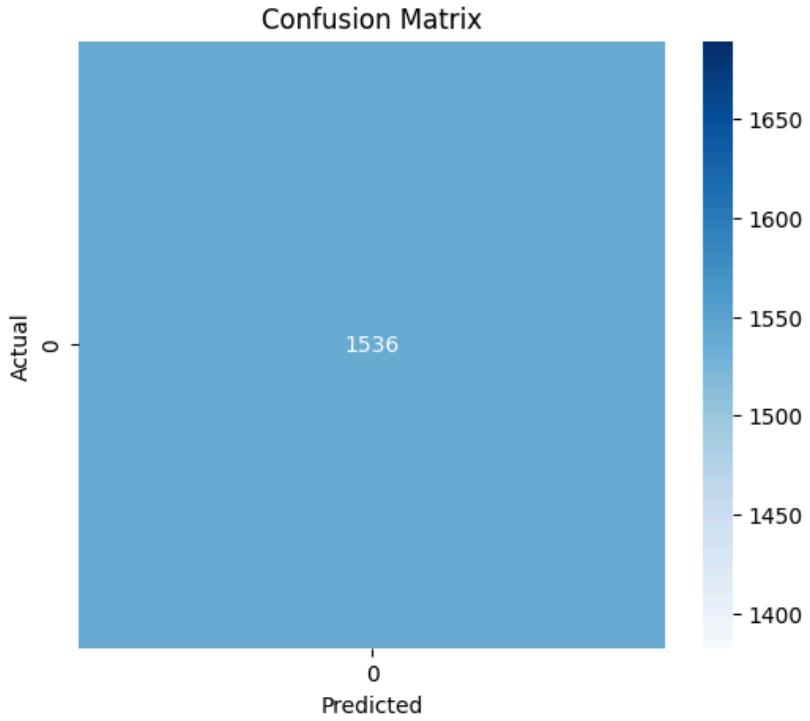
```
===== MODEL EVALUATION =====
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1536

    accuracy                           1.00      1536
   macro avg       1.00      1.00      1.00      1536
weighted avg       1.00      1.00      1.00      1536

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:407: UserWarning: A single
  warnings.warn(
```




```
# Feature importance
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("Feature Importance:")
print(importance)

# Plot feature importance
plt.figure(figsize=(10, 6))

importance.head(10).plot(kind='bar')

plt.title("Top Important Features")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Business insights
print("\n===== BUSINESS INSIGHTS =====")

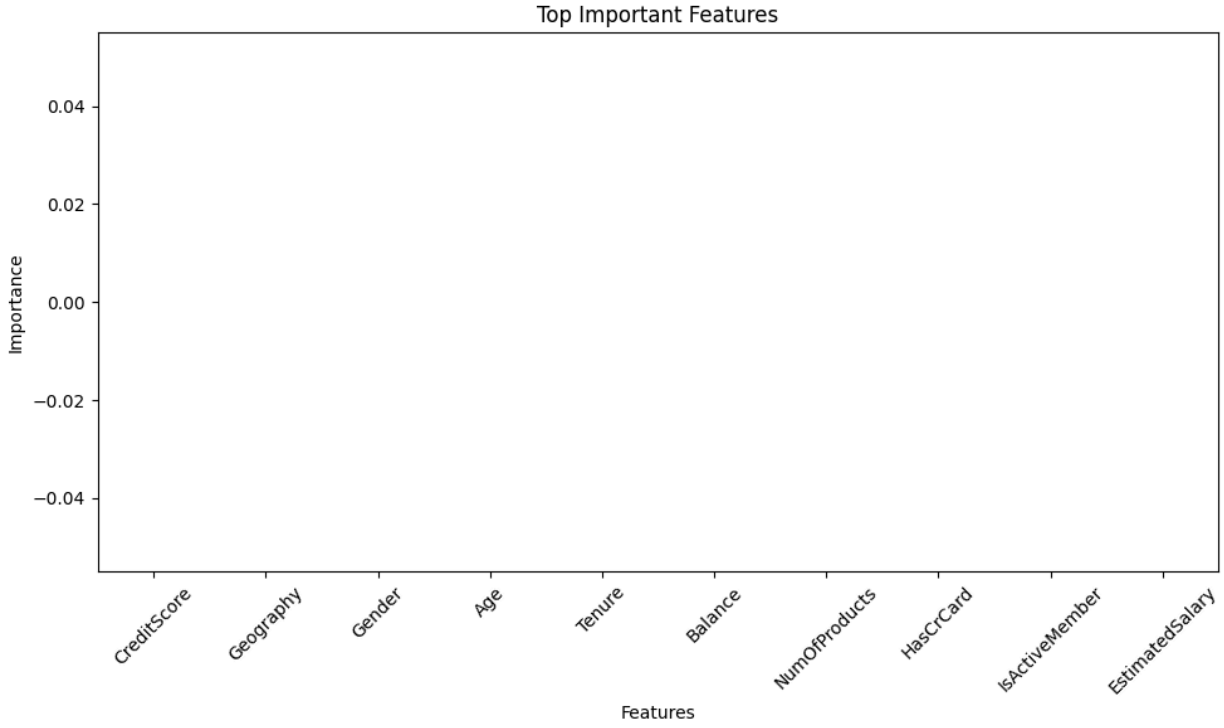
print("1. The most important features can strongly influence predictions.")
```

```
print("2. Customers with characteristics associated with higher churn")
print("   can be identified for targeted retention strategies.")

print("3. Model performance metrics indicate how reliably")
print("   the system predicts customer behavior.")

print("4. The analysis can help businesses identify opportunities")
print("   and potential risks.")
```

```
Feature Importance:
CreditScore        0.0
Geography          0.0
Gender             0.0
Age                0.0
Tenure             0.0
Balance            0.0
NumOfProducts      0.0
HasCrCard          0.0
IsActiveMember     0.0
EstimatedSalary    0.0
dtype: float64
```




```
===== BUSINESS INSIGHTS =====
1. The most important features can strongly influence predictions.
2. Customers with characteristics associated with higher churn
   can be identified for targeted retention strategies.
3. Model performance metrics indicate how reliably
   the system predicts customer behavior.
4. The analysis can help businesses identify opportunities
   and potential risks.
```

```
# Generate a simple project summary

print("==========================================")
print("       DATA ANALYTICS PROJECT REPORT")
print("==========================================")

print("\n1. DATASET")
print("Dataset was imported and analyzed using Python.")
```

```
print("\n2. PREPROCESSING")
print("Missing values, duplicate records, outliers,")
print("and categorical variables were handled.")

print("\n3. EDA")
print("Descriptive statistics and visualizations")
print("were used to understand the dataset.")

print("\n4. STATISTICAL ANALYSIS")
print("Correlation analysis was performed.")

print("\n5. MACHINE LEARNING")
print("A Decision Tree Classifier was trained.")

print("\n6. MODEL PERFORMANCE")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

print("\n7. CONCLUSION")
print("The developed analytics solution can support")
print("data-driven business decision making.")

print("\n========================================")
```

```
========================================
       DATA ANALYTICS PROJECT REPORT
========================================

1. DATASET
Dataset was imported and analyzed using Python.

2. PREPROCESSING
Missing values, duplicate records, outliers,
and categorical variables were handled.

3. EDA
Descriptive statistics and visualizations
were used to understand the dataset.

4. STATISTICAL ANALYSIS
Correlation analysis was performed.

5. MACHINE LEARNING
A Decision Tree Classifier was trained.

6. MODEL PERFORMANCE
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0

7. CONCLUSION
The developed analytics solution can support
data-driven business decision making.

========================================
```

```
print("========================================")
print("       FINAL ANALYTICS SOLUTION")
print("========================================")

print("\nDataset successfully analyzed.")

print("\nData preprocessing completed.")
print("EDA and visualization completed.")
print("Statistical analysis completed.")
print("Machine learning model developed.")
print("Model evaluation completed.")

print("\nModel Performance:")
```

```
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

print("\nBusiness Decision Support:")
print("1. Identify important factors affecting customers.")
print("2. Identify customers at higher risk of churn.")
print("3. Develop targeted customer retention strategies.")
print("4. Use analytical insights for better decision-making.")
print("5. Improve business performance using data-driven strategies.")

print("\n========================================")
print("          PROJECT COMPLETED")
print("========================================")
```

```
========================================
       FINAL ANALYTICS SOLUTION
========================================

Dataset successfully analyzed.

Data preprocessing completed.
EDA and visualization completed.
Statistical analysis completed.
Machine learning model developed.
Model evaluation completed.

Model Performance:
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0

Business Decision Support:
1. Identify important factors affecting customers.
2. Identify customers at higher risk of churn.
3. Develop targeted customer retention strategies.
4. Use analytical insights for better decision-making.
5. Improve business performance using data-driven strategies.

========================================
          PROJECT COMPLETED
========================================
```

THEORY QUESTIONS AND ANSWERS

Q1. What are the major stages involved in an end-to-end data analytics project?

Answer:

The major stages involved in an end-to-end data analytics project are:

1. Data collection and selection
2. Data preprocessing and cleaning
3. Exploratory Data Analysis (EDA)
4. Statistical analysis
5. Data visualization
6. Machine learning model development
7. Model evaluation
8. Interpretation of results
9. Identification of business insights
10. Preparation of reports and presentation

These stages help convert raw data into meaningful information that can support business decision-making.

Q2. Why is data preprocessing considered the foundation of successful analytics and machine learning?

Answer:

Data preprocessing is considered the foundation because real-world datasets may contain missing values, duplicate records, outliers, inconsistent data, and categorical variables.

Preprocessing improves the quality and reliability of the data before analysis and machine learning. It includes handling missing values, removing duplicates, treating outliers, and converting categorical data into a suitable numerical format.

Clean and properly prepared data helps machine learning models produce more accurate and reliable results.

Q3. How does Exploratory Data Analysis (EDA) contribute to model development?

Answer:

Exploratory Data Analysis helps understand the characteristics and patterns present in a dataset.

EDA uses descriptive statistics and visualizations to identify relationships between variables, distributions, trends, outliers, and unusual observations.

It helps in selecting important features, identifying problems in the data, and choosing an appropriate machine learning algorithm.

Therefore, EDA provides a better understanding of the dataset before developing the predictive model.

Q4. What factors should be considered while selecting a machine learning algorithm for a business problem?

Answer:

The following factors should be considered while selecting a machine learning algorithm:

1. Type of business problem
2. Type and size of the dataset
3. Nature of the target variable
4. Number and type of features
5. Required prediction accuracy
6. Interpretability of the model
7. Computational requirements
8. Training time
9. Available data quality
10. Suitable evaluation metrics

For example, classification algorithms can be used when the output belongs to different categories, while regression algorithms can be used for predicting continuous numerical values.

Q5. Why is model evaluation essential before deploying a predictive model?

Answer:

Model evaluation is essential because it determines how well a machine learning model performs on unseen data.

Metrics such as Accuracy, Precision, Recall, F1-Score, MAE, RMSE, and R^2 Score can be used depending on the type of problem.

Evaluation helps identify incorrect predictions and determines whether the model is reliable enough for practical use.

A model should be properly evaluated before deployment because poor predictions can lead to incorrect business decisions.

Q6. Explain how Business Intelligence tools complement machine learning in data analytics projects.

Answer:

Business Intelligence tools such as Tableau and Microsoft Power BI help present analytical results in the form of interactive dashboards, charts, and reports.

Machine learning focuses on prediction and finding patterns in data, while Business Intelligence helps users understand and communicate these results.

Together, they allow organizations to combine predictive analytics with visual reporting and support data-driven decision-making.

Q7. What challenges are commonly encountered while working with real-world datasets?

Answer:

Common challenges include:

1. Missing values
2. Duplicate records